MILESTONE 4 • MID-TERM PRESENTATION

智能体动态记忆管理

Agent Dynamic Memory Management

基于 LoCoMo 的结构化记忆构建与动态状态解析 Web 应用

SRE小组 • Team ID：第六组

成员：**王嘉淦　李胡诚　叶泽晨　马旻瑄　陈镇灏**

课程：Software Requirements Engineering

2026-10

项目背景与核心痛点

核心痛点：普通向量检索无法判断记忆是否过时，导致 AI 被历史状态误导。

① 痛点定位

记忆会变化：长程对话中，用户状态、计划和偏好持续更新。

仅靠相关性召回不够：普通向量检索只能找到“相关”信息，无法判断“是否仍有效”。

新旧状态混淆：系统容易同时召回当前状态与已被更新的历史状态，导致回答矛盾或错误。

② 典型反例

情景：Gina 1月失业于DoorDash，3月开了网店。

静态检索的缺陷：把“Gina 在 DoorDash 工作”当成当前事实召回，答错问题。

我们系统的解法：识别出两者是SUPERSEDE关系，以最新状态（开网店）作为有效记忆作答。

③ 影响范围

- 跨会话问答
- 连续项目跟进
- 角色扮演长线对话

普通 RAG 停留在“静态检索”，本应用升级为“状态演化解析”，能够判别过时、替代与冲突，定位当前有效记忆。

需求范围

必做 / Bonus | 实现状态

In-Scope（必做）

1. **数据导入与解析**：使用 6 段 LoCoMo 对话（4 段开发 + 2 段测试），解析原始多会话对话数据。 已实现
2. **结构化记忆抽取**：从对话中提取人物、时间等信息，构建包含内容、时间、来源会话的结构化 Memory。 已实现
3. **统一 JSON 导出**：支持将所有构建好的结构化 Memory 统一导出为 JSON 格式文件。 已实现
4. **Web 端记忆可视化**：在 Web 界面中展示构建的结构化 Memory 及其对应的原始对话证据。 已实现
5. **动态记忆状态解析**：针对输入检索相关 Memory，利用 LLM 判断记忆间的 RETAIN / UPDATE / SUPERSEDE / CONFLICT 关系。 已实现
6. **有效记忆问答与展示**：确定当前有效 Memory 并生成最终回答，Web 端同步展示检索证据与状态解析全过程。 已实现

Bonus

·复杂冲突消解与多版本保留	已实现
Graph Memory / 多跳关系建模	未实现
Memory 合并与压缩	部分实现
自适应检索 / 复杂时间推理	部分实现

系统架构

① 提问 → ② 接口转发 → ③ 检索+判断 → ④ 存取数据 → ③ 生成答案 → ① 展示

层	技术	作用
① 前端展示层	React	浏览记忆、提问、查看证据与答案
② API 服务层	FastAPI	接收请求、转发导入 / 构建 / 检索 / 问答接口
③ 检索与推理层	TF-IDF + 规则/LLM	找相关记忆 → 判断过时/替代/冲突 → 只保留有效记忆
④ 数据存储层	SQLite	存储对话、记忆与查询结果

关键设计点

- **可追溯防幻觉**：每条记忆带出处，答案能查回原文
- **固定四步**：粗筛 → 判关系 → 解析有效 → 生成答案
- **轻量易部署**：无重型向量库，SQLite + TF-IDF 即可跑
- **LLM 双模式**：无 Key 用 mock，配 Key 接真模型

项目结构

仓库目录 · 前端 · 后端 · 过程文档

backend/ • 后端 FastAPI

main.py 入口 · 路由 · 静态托管

models.py 数据契约 Memory / Relation

api/routes.py REST 端点

data/importer.py 原始对话导入

memory/ builder · extractor

retrieval/retriever.py TF-IDF 检索

reasoning/ 状态解析 4 模块

generation/ + llm/ 答案生成

export/exporter.py 统一 JSON 导出

tests/ pytest 35 项

frontend/src/ • 前端 React+TS

App.tsx • main.tsx 入口与装配

api/ client · types

MemoryBrowser 记忆列表 · 筛选

MemoryDrawer 详情 · 证据溯源

RelationTimeline 关系时间线

CandidateTable 候选 · 有效性

QueryWorkbench 问答工作台

ConversationBar • AppHeader 会话 · 顶栏

constants • hooks • utils 状态与工具

docs/process/ • 过程文档

meeting-minutes/ 3次例会纪要

plans/ 周计划 · 风险跟踪

design/ 系统设计文档

reviews/ SRS / 设计评审

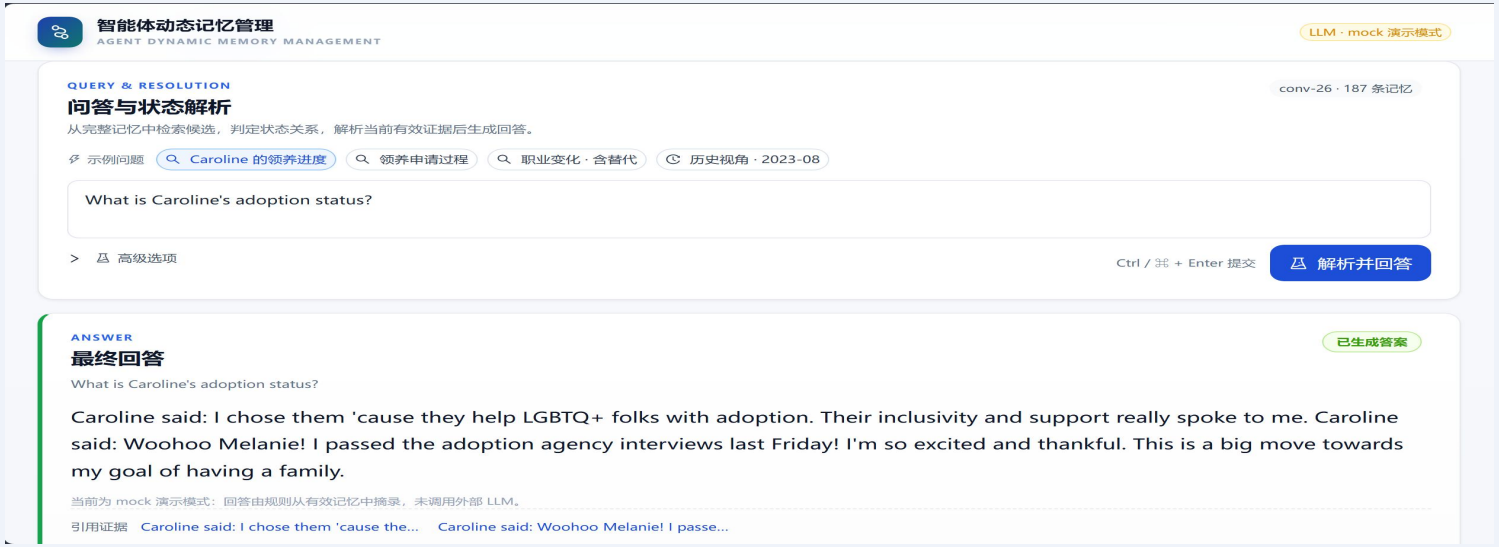
testing/ 测试报告 + 17 截图

worklog/ Git 提交记录

versions/ changelog + tag

代码仓库地址：https://github.com/w13126755377-ops/SRE-project

已实现功能一：记忆浏览与证据溯源





界面要素

列表 / 详情 / 证据面板

- 列表：内容 / 实体 / 属性 / 状态
- 详情抽屉：结构化字段 + 来源

交互能力

筛选 · 检索 · 展开来源

- 关键词搜索 · 实体 / 状态筛选
- 分页浏览，点行打开详情

证据溯源

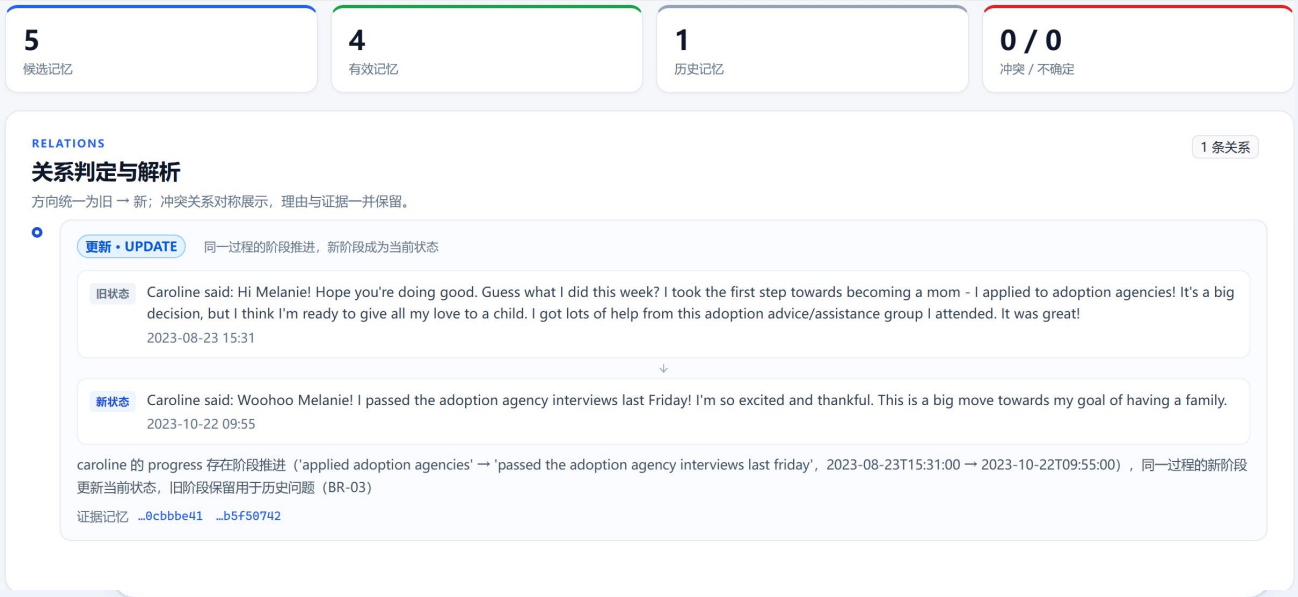
答案 → Memory → 原始对话轮次

- 每条记忆带来源轮次 turn_id
- 回溯 speaker / 时间 / 原文

已实现功能二：动态记忆问答

输入问题 → **检索历史 Memory** → **关系判断** → **有效记忆集合** → **生成答案**

用户提问 | TF-IDF 召回 Top-K | RETAIN / UPDATE / SUPERSEDE / CONFLICT | 只保留当前有效 | 带证据的回答



② 输出解释：为什么选这条记忆

- 逐条候选：相关性得分 + 有效性状态
- 关系：RETAIN / UPDATE / SUPERSEDE / CONFLICT
- 有效性：有效 / 历史 / 冲突 / 不确定
- 冲突消解：轮次自纠 / 来源 / 证据量
- 回答只引用有效证据，可回查来源

核心机制：检索 + 关系判断 → 输出「当前有效状态」，而不是「最相似的历史片段」

测试结果

17/17	35	369/369	187
场景通过	pytest 全部通过	证据可追溯 · 0 失败	JSON 导出 · 全带来源

已验证的完整链路

17 场景 · 15 通过 · 2 未覆盖

① 导入 → ② 构建 → ③ 溯源 → ④ 检索 → ⑤ 判定 → ⑥ 有效性解析 → ⑦ 回答/引用 → ⑧ 导出

限制

· 批量评测（/api/eval 暂用501占位）

· 模型注入未执行

数据与配置

· demo-2conv：conv-26 / conv-30

· 38 会话 / 788 轮 / 304 QA

· mock + rules · TF-IDF 检索

口径：M4 功能测试报告（2026-10-21）· 前端 tsc 与构建通过 · 导出 187 条全带来源 · schema_version 0.1

待改进

1 语义近义词关系

只按字面匹配，无法识别同义 / 近义表达 —— 如 'job' 与 'occupation' 被当成无关词。

改进：换向量召回，或加同义词表做查询扩展。

2 忽略词序、语法、否定

词袋模型丢弃语序与语法结构，否定词无法区分 —— 如 'not happy' 与 'happy' 被看作相似。

改进：引入 n-gram / 句法特征，或用 LLM 对候选重排并显式识别否定词。

3 词形变化、拼写、缩写不鲁棒

对时态变形、拼写错误、缩写敏感 —— 如 run / runs / running 视为三个词，'US' 与 'United States' 失配。

改进：检索前做词干stemming、词形还原、拼写纠错与缩写扩展。

总结

① 做成了什么

完成必做闭环

6 项必做全部实现：导入 → 构建 → 浏览溯源 → 检索 → 状态解析 → 问答导出。

攻克bonus核心难点

动态状态解析，包括复杂冲突消解与多版本保留，只保留当前有效记忆，避免被过时历史误导。

验证系统有效

15/17 场景通过 · 35 项 pytest · 369/369 证据可回溯 · 187 条导出全带来源。

② 下一步

批量评测

落地 FR-13：计算 F1 / Recall@K / 状态准确率，对比 Full Text / Vector RAG 基线。

换向量召回

替代 TF-IDF，改善语义近义、词序语法否定、词形拼写缩写等局限。

图记忆与冲突消解

Graph Memory 多跳关系建模；完善 CONFLICT 自动消解。

SRE小组•谢谢聆听

感谢观看 · 欢迎提问